SEMESTER PROJECT PROPOSAL

Smart Blood Bank Management System

Expiry-Aware Allocation with Blood-Group Compatibility Matching

Item	Details
Course	Software Construction and Development Lab (SEN-311L)
Project type	Web Application (Complex Computing Problem)
Target users	Blood bank staff, laboratory technicians, hospital requesters, blood donors, and system administrators
Primary CCP characteristics	Conflicting Requirements, Extent of Stakeholder Involvement, Consequences
Supporting CCP characteristics	Interdependence, Depth of Analysis

Project Team

Team Member	Roll Number
Muhammad Abbas Haider	137/F24/BSSE/5C
Asadullah	120/F24/BSSE/5C
Muneeb Ejaz	144/F24/BSSE/5C
Syed Saif	161/F24/BSSE/5C
Ibrahim Asif	142/F24/BSSE/5C

1. Introduction

Blood is a perishable, life-saving resource that cannot be manufactured. Every unit has a limited shelf life, must match the recipient's blood group, and is often needed urgently. Blood banks must therefore decide, many times a day, which unit to issue, to whom, and in what order.

The Smart Blood Bank Management System is a web application that goes beyond record keeping. In addition to donor management, inventory tracking, and request processing, it includes an allocation engine that recommends which compatible blood units to issue, based on blood-group compatibility, expiry date, request urgency, and the scarcity of universal-donor stock. Staff keep the final decision; the system makes the decision faster, safer, and less wasteful.

2. Problem Statement

Many blood banks track donors, stock, and requests in notebooks or spreadsheets. This causes four specific problems:

- **No real-time visibility:** staff cannot see current stock by blood group without cross-checking several records.
- **Expiry wastage:** expired bags go unnoticed until a physical check, so scarce blood is wasted.
- **Unsafe or inefficient matching:** choosing a compatible unit manually under time pressure risks errors, or wastes universal-donor blood (O-negative) on patients who did not need it.
- **Scattered history:** fulfilment and audit records are spread across separate sources, so accountability is weak.

The core problem is that the blood bank must satisfy competing goals at the same time: respond to emergencies quickly, issue only safe and compatible units, minimize expiry wastage, and preserve scarce stock. No single rule satisfies all of these, which is what makes this a Complex Computing Problem.

3. Objectives

- Give staff real-time visibility into blood stock by group, including units nearing expiry.
- Recommend compatible units for each request, ranked using a transparent, configurable scoring policy.
- Reduce expiry wastage without slowing down emergency fulfilment.
- Support the different roles involved (staff, lab technicians, hospital requesters, administrators) with role-based access.
- Block unsafe actions (issuing an expired or incompatible unit) and record every action in an audit trail.
- Deliver a working core system within the 6-week academic schedule, structured so that donor reminders and hospital-network integration can be added later.

4. Complex Computing Problem (CCP) Justification

This project is not a standard record-keeping application. The complexity comes from the trade-offs, stakeholders, and consequences described below, not from the number of features. The table summarizes each CCP characteristic the project satisfies.

CCP Characteristic	Evidence in this Project	How the Project Addresses It
1. Conflicting Requirements (Primary)	Speed vs. wastage: emergencies need the fastest available unit, while wastage reduction needs the oldest unit to be issued first. Safety vs. availability: strict group matching is safest, but allowing compatible substitutes keeps more requests fulfilled. Preservation vs. demand: O-negative is compatible with everyone, so using it freely depletes the stock needed for true emergencies. Usability vs. security: staff need quick access, but donor and patient data require authentication and role checks.	A weighted scoring policy balances these goals, and urgency changes the weights (Critical requests favor speed and compatibility; Routine requests favor expiry and preservation of universal stock). The policy is evaluated by simulation (see Section 7).

CCP Characteristic	Evidence in this Project	How the Project Addresses It
2. Extent of Stakeholder Involvement (Primary)	Blood bank staff: record donations and process requests. Laboratory technicians: test and clear units before they can be issued. Hospital requesters: submit requests with an urgency level and track status. Administrators: manage users, thresholds, and reports. Donors: provide data that must stay private and accurate.	Each group has different workflows, permissions, and privacy needs. Role-based access control restricts every screen and action to the roles that need it, and the unit lifecycle (Quarantined, Available, Reserved, Issued, Expired) reflects the lab's role in clearing blood.
3. Consequences (Primary)	Issuing an incompatible or expired unit is a patient-safety risk. Stock-outs of a critical group can delay treatment. Expired units are wasted blood, which is a scarce donated resource. A leak of donor data creates privacy and legal exposure.	Hard rules block expired or incompatible issue. Staff must confirm every allocation, and the system is decision support only; laboratory cross-matching remains mandatory. All actions are audit-logged, passwords are hashed, and data is filtered by institution.
4. Interdependence (Supporting)	Inventory, request management, the allocation engine, alerts, the dashboard, and the audit log depend on each other.	Issuing one unit changes stock, which changes alerts, dashboard totals, and the audit trail. Allocation is performed as one atomic database transaction so these components never become inconsistent.
5. Depth of Analysis (Supporting)	There is no single correct allocation rule; alternatives must be compared.	The team will compare FIFO, FEFO (first-expiry-first-out), and the weighted-priority policy using measurable criteria: wastage rate, exact-match rate, decision time, and zero compatibility violations.

5. Benchmarking with Existing Systems

The table compares the proposed system with the approaches available to Pakistani blood banks and hospitals today: manual tracking, a provincial regulator's donor app, a local donor-finder app, and a commercial hospital transfusion system. Entries are based on publicly described features. "Not stated" means the capability is not described in the public material reviewed.

Feature / Aspect	Manual / Spreadsheet	SBTA Donor App (Sindh)	Blood Bank Pakistan App	WellSky Transfusion (commercial)	Proposed System
Real-time stock by blood group	No	Limited	Not stated	Yes	Yes
Donor search and donation history	No	Yes	Not stated	Not stated	Yes
Expiry tracking and alerts	No	Not stated	Not stated	Yes	Yes
Expiry-aware (FEFO) allocation suggestions	No	Not stated	Not stated	Limited	Yes

Feature / Aspect	Manual / Spreadsheet	SBTA Donor App (Sindh)	Blood Bank Pakistan App	WellSky Transfusion (commercial)	Proposed System
Blood-group compatibility matching	Manual	Not stated	Not stated	Not stated	Yes
Urgency-based prioritization	No	Not stated	Not stated	Limited	Yes
Preservation of universal-donor (O-) stock	No	Not stated	Not stated	Not stated	Yes
Audit trail of all actions	Limited	Not stated	Not stated	Not stated	Yes
Simple setup for a single facility	Yes	Yes	Yes	No (enterprise)	Yes

Local regulatory context: the Punjab Blood Transfusion Authority's standards for blood establishments call for every blood unit to be identifiable and traceable from donor to recipient, and for a current inventory to be kept for each storage category. The proposed system's audit trail and inventory modules are designed to support these expectations.

Gap identified: the local tools reviewed focus on connecting donors with blood banks, while the commercial system is built for large hospital laboratories. Neither is described as helping a small blood bank choose which compatible unit to issue by balancing expiry, urgency, and scarcity. The proposed system targets that gap in a lightweight form.

6. Proposed System Features and Architecture

6.1 Core Modules

Module	Description
Authentication and Roles	Registration, login, logout, and role-based access for Admin, Staff, Lab Technician, and Hospital Requester.
Donor Management	Donor profiles with blood group, contact details, and last-donation date used for eligibility checks.
Inventory Management	Donation records with automatic expiry date; unit status lifecycle (Quarantined, Available, Reserved, Issued, Expired).
Request Management	Requests with blood group, quantity, and urgency (Routine, Urgent, Critical); status tracking (Pending, Fulfilled).
Compatibility and Allocation Engine	Filters compatible, unexpired, cleared units and ranks them by a configurable score. Staff confirm the final allocation.
Alerts	Low-stock alerts per blood group and near-expiry alerts, using configurable thresholds.
Dashboard and Reports	Total bags and breakdown by blood group, near-expiry count, and a wastage summary.
Audit and Transaction History	Immutable log of who changed what and when, including every allocation.

6.2 Blood-Group Compatibility Rules (Red Cells)

The engine uses the following standard red-cell compatibility table to filter candidate units for a recipient.

Recipient Group	Compatible Donor Groups	Exact Match Preferred
O-	O-	O-
O+	O+, O-	O+
A-	A-, O-	A-
A+	A+, A-, O+, O-	A+
B-	B-, O-	B-
B+	B+, B-, O+, O-	B+
AB-	AB-, A-, B-, O-	AB-
AB+	All eight groups	AB+

6.3 Allocation Logic

- **Step 1, filter:** keep only units that are Available, not expired, and compatible with the recipient group.
- **Step 2, score:** rank units by (a) match quality (exact match scores highest), (b) closeness to expiry (sooner expiry scores higher), and (c) a scarcity penalty for using O-negative on non-O-negative recipients.
- **Step 3, adjust by urgency:** Critical requests weight match quality and availability most; Routine requests weight expiry and stock preservation most.
- **Step 4, confirm and commit:** staff review the ranked suggestions and confirm; the allocation, stock update, and audit entry are saved in one atomic transaction.

6.4 System Architecture

Layer	Responsibility
Presentation Layer	Browser-based interface: forms, dashboard, alerts, and request screens, with role-specific views.
Application Layer	Authentication, role checks, business rules, the allocation engine, alert generation, and input validation.
Data Layer	Relational database holding users, donors, blood units, requests, transactions, and audit logs.

Core data entities: User (with role), Donor, Blood Unit, Request, Transaction, and Audit Log. One donor can give many units; one request can be fulfilled by several units; every transaction links a request to a unit.

7. Analysis and Validation Plan

Because there is no single correct allocation rule, the team will compare three strategies on the same simulated dataset of donations and requests across a 60-day period.

Strategy	Description	Expected Trade-off
FIFO	Issue the unit collected earliest.	Simple, but ignores match quality and scarcity.
FEFO	Issue the unit expiring soonest among compatible units.	Low wastage, but may use scarce O- stock too freely.
Weighted priority (proposed)	Score by match quality, expiry, scarcity, and urgency.	Balances all goals; needs tuned weights.

Evaluation criteria: percentage of units expired, percentage of requests served with an exact match, O- negative units used for non-O-negative recipients, decision time per request, and compatibility violations (target: zero).

8. Key Requirements

8.1 Functional Requirements

ID	Requirement
FR-01	The system shall allow a new user to register with a username, email address, and password.
FR-02	The system shall allow a registered user to log in and log out.
FR-03	The system shall restrict each screen and action to the user's role (Admin, Staff, Lab Technician, Hospital Requester).
FR-04	The system shall allow staff to create, edit, and delete donor profiles, and shall record the last-donation date.
FR-05	The system shall allow staff to log a donation with donor, blood group, and collection date, and shall calculate the expiry date automatically.
FR-06	The system shall allow a lab technician to mark a unit as cleared before it becomes Available.
FR-07	The system shall allow authorized users to edit or delete an inventory record and shall log the change.
FR-08	The system shall allow a requester to submit a request with blood group, quantity, and urgency level.
FR-09	The system shall list compatible, unexpired, cleared units for a request, ranked by the allocation score.
FR-10	The system shall block the issue of any expired or incompatible unit.
FR-11	The system shall allow staff to confirm an allocation and mark the request Fulfilled or Pending, updating stock in one atomic transaction.
FR-12	The system shall raise low-stock and near-expiry alerts using administrator-set thresholds.
FR-13	The system shall display total blood bags, a breakdown by blood group, and near-expiry units on the dashboard.
FR-14	The system shall allow authorized users to view the transaction and audit history.
FR-15	The system shall provide a summary report of units issued, expired, and wasted per period.
FR-16	The system shall allow a user to update their own profile.

8.2 Non-Functional Requirements

ID	Requirement
NFR-01	Usability: a first-time user shall be able to register, add a record, and mark a request fulfilled without training.
NFR-02	Performance: standard actions, including allocation suggestions, shall respond within 3 seconds under normal load.
NFR-03	Security: passwords shall be stored with a secure hashing algorithm, and authentication shall be required before any data is displayed.
NFR-04	Reliability: saved inventory and transaction records shall not be lost or corrupted during normal operation.
NFR-05	Maintainability: the system shall be built as separate, documented modules.
NFR-06	Compatibility: the application shall work on current versions of Chrome, Firefox, and Edge.
NFR-07	Privacy: a user shall access and modify only their own institution's data.
NFR-08	Auditability: every create, edit, delete, and allocation action shall be recorded with user and timestamp.

9. Tools and Technologies

Area	Proposed Technology
Frontend	HTML, CSS, JavaScript (Chart.js for dashboard charts)
Backend	A single server-side technology chosen during design, e.g., Node.js with Express, PHP, or Python
Database	MySQL or SQLite (final choice during design)
Security	bcrypt-style password hashing, session or token-based authentication
Modeling and Design	UML diagrams (use case, class, sequence); ER diagram for the database
Testing	Unit and integration tests for the allocation engine; simulation scripts for strategy comparison
Version Control	Git and GitHub

10. Timeline

Phase	Duration	Deliverables
Requirements, CCP analysis, and benchmarking	Week 1	Finalized requirements, CCP justification, benchmarking table
System design and database planning	Week 2	UML diagrams, ER diagram, allocation-algorithm design
Development	Weeks 3 to 4	Authentication and roles, inventory, requests, allocation engine, dashboard
Testing and simulation	Week 5	Test cases per requirement; FIFO vs. FEFO vs. weighted comparison

Phase	Duration	Deliverables
Final documentation and presentation	Week 6	Final report and presentation

11. Risks and Mitigation

Risk	Mitigation
Scope expansion	Limit the first release to the requirements in Section 8; defer extras to future enhancements.
Allocation weights are hard to tune	Make weights configurable and tune them using the simulation results.
Incorrect compatibility logic	Test every recipient and donor group pair against the compatibility table.
Database errors or data loss	Use atomic transactions and daily backups during development.
Development delays	Assign tasks per member and review progress weekly against the timeline.
Unfamiliar technology	Choose technologies at least one team member already uses and document setup steps.

12. Scope and Future Enhancements

In scope: the modules and requirements listed above, delivered as a web application.

Out of scope for the first release: AI-based demand prediction, wearable-device integration, direct hospital-network integration, cloud synchronization, and a native mobile application.

Note on safety: the system provides decision support only. It does not replace laboratory testing or cross-matching, which remain mandatory before any transfusion.

13. Conclusion

The Smart Blood Bank Management System addresses a Complex Computing Problem: balancing speed, safety, wastage, and scarcity across several stakeholder groups, where an incorrect decision can harm patients. By combining inventory management with an expiry-aware, compatibility-based allocation engine, and by validating the allocation policy through simulation, the project moves well beyond routine record keeping while remaining achievable within one semester.